

SKYSPICE2KLAYOUT AnalogProPlus v12

My AI-assisted SKY130 SPICE-to-KLayout helper flow for analog layout exploration.

I built this as a summer hobby project because I was frustrated with the usual `import spice` experience while playing with SKY130 analog circuits. I wanted a cleaner starting point inside KLayout: parse a SPICE/CDL netlist, generate or import the devices, place them in a useful way, write reports, and give me enough visual context to continue manual layout work.

This is not a commercial EDA tool, not a signoff flow, and not a replacement for analog layout skill. I still treat manual review, Magic DRC, extraction, LVS, and foundry rule checks as the real checks. This project is a playground for making SPICE-to-GDS exploration less painful.

Contents

- [Project scope](#project-scope)
- [Internal flow](#internal-flow)
- [Analog intuition behind the flow](#analog-intuition-behind-the-flow)
- [Why hybrid mode exists](#why-hybrid-mode-exists)
- [Repository layout](#repository-layout)
- [Files and libraries used](#files-and-libraries-used)
- [Expected paths](#expected-paths)
- [Why I use klayout_gf7](#why-i-use-klayout_gf7)
- [Setup](#setup)
- [Doctor check](#doctor-check)
- [Command syntax](#command-syntax)
- [Run folder behavior](#run-folder-behavior)
- [Run modes](#run-modes)
- [Wizard mode](#wizard-mode)
- [Command examples](#command-examples)
- [Generated outputs](#generated-outputs)
- [Troubleshooting](#troubleshooting)
- [Validation order](#validation-order)
- [Final note](#final-note)

Project scope

The flow is meant to create a useful first layout view from a SKY130 SPICE/CDL netlist. It is not meant to finish analog layout automatically.

Area	What v12 does
Netlist input	Parses SKY130 SPICE/CDL-style netlists and extracts devices, nodes, model names, and common parameters.
Device detection	Classifies MOS, BJT, diode, capacitor, resistor, standard-cell-like, IO-like, and fixed-device entries.
MOS generation	Supports gdsfactory, magic, and hybrid backends.
Fixed devices	Imports fixed GDS cells such as BJTs and other primitive SKY130 cells when available.
Placement	Supports analogpro topology-aware placement and schematic SPICE-order placement.
Reports	Writes device maps, net maps, topology reports, routing-priority reports, warnings, unit-array reports, and run dashboards.
Analog helpers	Adds unit-array planning, common-centroid markers, dummy markers, topology hints, and routing-intent reports.
Ratline reference	Can generate separate _ratpoints.gds and _ratlines_full.gds files on non-fab/debug layers.
Output handling	Keeps every run in a timestamped folder under ./skyspice_runs/.

User interface	Supports command-line mode and interactive wizard mode.
----------------	---

What this project is not

Not this	What I mean
Not a production analog layout generator	It gives a structured starting point. It does not create final analog layout.
Not a router	Ratlines are visual reference geometry, not routed metal.
Not a signoff tool	Magic DRC and LVS helpers are useful smoke checks, but final verification is still manual and tool-chain dependent.
Not a foundry-rule replacement	PDK rules, manual layout review, DRC, extraction, and LVS still matter.
Not a universal SKY130 installer	It assumes SKY130 PDK, KLayout, Magic, Netgen, and the gf7 Python environment are installed correctly.

Internal flow

At a high level, the tool runs like this:

SPICE/CDL netlist -> launcher sets PDK and Python paths -> KLayout batch mode loads the Python script -> script parses devices and nets -> devices are generated or imported through Magic, gdsfactory, fixed GDS, or libs.ref fallback -> placement is created using analogpro or schematic mode -> optional helpers add unit-array planning, markers, ratline reference files, DRC/LVS helper logs -> all outputs are written into a clean timestamped run folder
--

Stage	What happens	Why it matters
Launcher setup	Sets PDKPATH, KLAYOUTH_PATH, KLAYOUTH_PY_SITE, PYTHONPATH, and command options.	Most failures came from wrong paths, so the launcher is part of the flow, not just a wrapper.
Netlist parsing	Reads SPICE/CDL devices, instance names, nodes, W/L/nf/m values, and model names.	This decides what needs to be generated, imported, or reported as missing.
Device generation	Uses Magic, gdsfactory, fixed GDS, or library fallback depending on backend and device type.	Different device types need different strategies. Guard-ring MOS and normal MOS are not treated the same.
Placement	Uses analogpro or schematic mode.	I can either get a topology-aware starting point or a simple netlist-order view.
Reports	Writes CSV/JSON reports, warnings, routing priority, unit-array plans, and topology summaries.	The reports make the run debuggable instead of only producing a GDS.
Optional views	Ratpoints and ratlines are written as separate GDS files.	I can inspect connectivity without polluting the clean main GDS.

Analog intuition behind the flow

Analog layout is not only about placing shapes. Matching, symmetry, common-centroid thinking, dummy environment, body ties, guard rings, routing parasitics, device orientation, and local neighborhood all affect the final circuit. This script does not solve all of that, but it tries to organize the first messy step so the layout review starts from something understandable.

Analog idea	How I represent it in the tool	Important limitation
Large MOS devices often need unit thinking	SKY_UNIT_ARRAY_MODE=plan writes unit-array plans. layout mode can replace large devices with unit-array wrappers.	Strict LVS may see many parallel MOS devices instead of one large MOS.
Current mirrors should be visible early	Topology reports and placement hints try to group mirror-like devices.	This is a hint, not a final matched layout.
Sensitive nets should be obvious	Routing-priority reports flag important nets and estimated connectivity pressure.	It does not route the net.
Common-centroid starts with grouping	SKY_CC_MODE=markers adds non-fab planning markers and writes a plan file.	The marker is not a finished common-centroid layout.

Dummy devices should not be blindly inserted	SKY_DUMMY_MODE=marker adds dummy planning markers only.	Real dummy MOS insertion affects LVS and needs connection strategy.
Guard rings are real physical layout	Selected guard-ring devices are generated through Magic in hybrid mode.	Blind guard-ring generation can create DRC or spacing problems.
Sometimes schematic order is easier	--placement schematic follows the netlist order more directly.	It is simpler, not smarter.
Sometimes topology-aware placement helps	--placement analogpro tries analog grouping and layout hints.	It is still a starting point, not final placement.

The practical goal is simple: the generated GDS should help me think about the analog layout, not pretend that layout is already complete.

Why hybrid mode exists

This part matters because it came directly from the problems I hit while testing.

When I tried using the native SKY130 KLayout Python `draw_fet.py` guard-ring path for all guard-ring devices, I ran into DRC-style problems around guard-ring geometry, tap/well behavior, and device spacing. I did not want generated guard rings if they were going to create new layout problems.

Magic already has a SKY130 generation/import-style flow that is closer to what Magic itself expects. So my practical split became:

selected MOS devices needing guard rings -> Magic backend
normal MOS devices without guard rings -> gdsfactory / SKY130 Python draw files
fixed devices like BJT -> fixed GDS import

That is why `hybrid` exists. I do not treat one backend as always better. I use Magic where guard-ring behavior matters and gdsfactory/SKY130 Python helper cells where normal MOS direct generation is enough.

Repository layout

This is the folder layout I am using for the project folder:

Analogprov12plus_runs/
├─ README.md
├─ required_docs/
│ └─ AnalogProPlus_v12_User_Guide.docx
│ └─ AnalogProPlus_v12_User_Guide.pdf
├─ required_scripts/
│ └─ skyspice2klayout_all_devices_magicgr_analogproplus.py
│ └─ skyspice2klayout_all_magicgr_analogproplus
│ └─ optional_sky130_helper_draw_files/
└─ skyspice_runs/
└─ BGR_sky130_ckt.spice

For local use, I install the script and launcher here:

~/ASIC_eda/klayout_scripts/
~/ASIC_eda/bin/

The project folder is for keeping the files organized. The `~/ASIC_eda` folders are where I actually run the tool from the terminal.

Files and libraries used

Project files

File or folder	Role
required_scripts/	Main Python script loaded by KLayout batch mode. It parses the

skyspice2klayout_all_devices_magicgr_analogproplus.py	netlist, generates/imports cells, places devices, and writes reports.
required_scripts/skyspice2klayout_all_magicgr_analogproplus	Launcher script. It sets PDK paths, forces the gf7 Python environment, handles CLI flags, and calls KLayout.
required_scripts/optional_sky130_helper_draw_files/	Optional backup/helper draw files. I only use these if the local PDK helper files are missing or broken.
required_docs/	DOCX and PDF user guide.
skyspice_runs/	Generated run folders. This can get large because every run writes GDS, JSON, CSV, logs, and dashboard files.

SKY130/Open PDK files used

PDK file or folder	Why it matters
/usr/local/share/pdk/sky130A/libs.tech/klayout/python/cells	Main SKY130 KLayout Python helper cell folder. MOS/cap/res generation helpers usually live here.
/usr/local/share/pdk/sky130A/libs.tech/klayout/pymacros/cells	Alternate helper root. Different open_pdk/KLayout installs may place helper code here.
/usr/local/share/pdk/sky130A/libs.tech/klayout/python/cells/fixed_devices	Fixed GDS cells for primitives such as BJTs and other fixed-layout devices.
/usr/local/share/pdk/sky130A/libs.ref	Fallback search location for library GDS cells, standard-cell-like content, and other referenced layouts.
/usr/local/share/pdk/sky130A/libs.tech/magic/sky130A.magicrc	Magic rcfile used for Magic generation, DRC helper, and extraction helper flows.

Python/runtime pieces

Piece	Use in the flow
KLayout pya API	Creates cells, reads/writes GDS, inserts labels, creates debug layers, and builds the top layout.
gdsfactory 7.x	Used by the SKY130 Python draw helper path for direct MOS/device generation.
gdstk	Used by gdsfactory and GDS geometry workflows.
PyYAML	Used when constraints or YAML-style configuration files are enabled.
Python standard library	Used for paths, subprocesses, temporary files, JSON/CSV reports, regular expressions, matching, and environment handling.

The helper/draw files are the bridge between a SPICE device name and actual generated geometry. I try to use the installed PDK helper files first. I only copy helper files from the repository if the local PDK install is incomplete.

Expected paths

My working setup uses this path layout:

Item	Expected path
PDK root	/usr/local/share/pdk
SKY130 PDK	/usr/local/share/pdk/sky130A
KLayout tech	/usr/local/share/pdk/sky130A/libs.tech/klayout
KLayout Python helpers	/usr/local/share/pdk/sky130A/libs.tech/klayout/python
KLayout pymacros helpers	/usr/local/share/pdk/sky130A/libs.tech/klayout/pymacros
Fixed-device GDS	/usr/local/share/pdk/sky130A/libs.tech/klayout/python/cells/fixed_devices
Magic rcfile	/usr/local/share/pdk/sky130A/libs.tech/magic/sky130A.magicrc
libs.ref	/usr/local/share/pdk/sky130A/libs.ref
gdsfactory venv	~/klayout_gf7

If your PDK is somewhere else, update environment variables first:

```
export PDK_ROOT=/your/pdk/root
export PDK=sky130A
export PDKPATH="$PDK_ROOT/$PDK"
```

Then check the setup:

```
skyspice2klayout_all_magicgr_analogproplus --doctor
```

If needed, update only the launcher defaults:

```
nano ~/ASIC_eda/bin/skyspice2klayout_all_magicgr_analogproplus
```

Look for:

```
PDK_ROOT="${PDK_ROOT:-/usr/local/share/pdk}"  
PDK="${PDK:-sky130A}"  
PDKPATH="${PDKPATH:-$PDK_ROOT/$PDK}"
```

Change the default path only if your install is not under `/usr/local/share/pdk`.

Why I use klayout_gf7

The gdsfactory side of this flow is sensitive to the gdsfactory/KFactory API version. I hit errors like:

```
cellname dev_temp already exists  
Cell object has no attribute add_array  
Component.add_ref() got unexpected keyword argument spacing  
list object has no attribute move
```

Those errors came from version/API mismatches between older SKY130 helper scripts and newer gdsfactory/KFactory behavior.

The stable solution I use is a separate virtual environment in my home folder:

```
~/klayout_gf7
```

The launcher forces KLayout to use this venv instead of accidentally loading packages from:

```
~/local/lib/python3.12/site-packages
```

That `local` fallback caused a lot of confusing behavior, so the launcher blocks user-site packages and forces the gf7 site-packages path.

Setup

1. Create the klayout_gf7 environment

Run this once:

```
python3 -m venv ~/klayout_gf7  
~/klayout_gf7/bin/python -m pip install --upgrade pip setuptools wheel  
~/klayout_gf7/bin/pip install "gdsfactory==7.*" gdstk shapely numpy scipy pyyaml
```

Verify it directly:

```
~/klayout_gf7/bin/python - <<'PY'  
import sys  
print("python:", sys.executable)  
import gdsfactory as gf  
print("gdsfactory:", gf.__version__)
```

```
print("gdsfactory file:", gf.__file__)
import gdstk
print("gdstk:", gdstk.__version__)
PY
```

Expected idea:

```
python: /home/<user>/klayout_gf7/bin/python
gdsfactory: 7.x.x
gdsfactory file: /home/<user>/klayout_gf7/lib/python3.12/site-packages/...
```

If this shows packages from `~/local`, fix the environment before running the layout flow.

2. Install the script and launcher

From the project root:

```
mkdir -p ~/ASIC_edu/klayout_scripts ~/ASIC_edu/bin

cp required_scripts/skyspice2klayout_all_devices_magicgr_analogproplus.py \
~/ASIC_edu/klayout_scripts/skyspice2klayout_all_devices_magicgr_analogproplus.py

cp required_scripts/skyspice2klayout_all_magicgr_analogproplus \
~/ASIC_edu/bin/skyspice2klayout_all_magicgr_analogproplus

chmod +x ~/ASIC_edu/bin/skyspice2klayout_all_magicgr_analogproplus
```

Add the launcher folder to the shell path:

```
export PATH="$HOME/ASIC_edu/bin:$PATH"
hash -r
```

Optional permanent shell setup:

```
echo 'export PATH="$HOME/ASIC_edu/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Check which launcher is being used:

```
type -a skyspice2klayout_all_magicgr_analogproplus
```

The first result should be:

```
/home/<user>/ASIC_edu/bin/skyspice2klayout_all_magicgr_analogproplus
```

3. Launcher path block

The launcher should force the gf7 environment. The important part should look like this:

```
GF7_PY="$HOME/klayout_gf7/bin/python"

KLAYOUT_PY_SITE="$($GF7_PY - <<'PY'
import site
print(site.getsitepackages()[0])
PY
)"

export KLAYOUT_PY_SITE
export PYTHONNOUSERSITE=1
export KLAYOUT_PATH="$PDKPATH/libs.tech/klayout"
export PYTHONPATH="$KLAYOUT_PY_SITE:$PDKPATH/libs.tech/klayout/python:$PDKPATH/libs.tech/klayout/pymacros:${PYTHONPATH:-}"
```

This block is important. It keeps KLayout from loading the wrong gdsfactory installation.

4. Optional helper draw files

The normal source for SKY130 helper files should be the installed PDK:

```
/usr/local/share/pdk/sky130A/libs.tech/klayout/python/cells
/usr/local/share/pdk/sky130A/libs.tech/klayout/pymacros/cells
```

If those are missing or broken, I keep optional known-working helper draw files in:

```
required_scripts/optional_sky130_helper_draw_files/
```

Back up the PDK helper folder before copying anything:

```
PDKPATH=/usr/local/share/pdk/sky130A

sudo cp -a "$PDKPATH/libs.tech/klayout/python/cells" \
"$PDKPATH/libs.tech/klayout/python/cells.backup_$(date +%Y%m%d_%H%M%S)"
```

Copy only if needed:

```
sudo cp -r required_scripts/optional_sky130_helper_draw_files/cells/* \
/usr/local/share/pdk/sky130A/libs.tech/klayout/python/cells/
```

If the current setup works, I do not touch this.

Doctor check

Always run this first:

```
skyspice2klayout_all_magicgr_analogproplus --doctor
```

Expected important lines:

```
KLAYOUT_PATH   : /usr/local/share/pdk/sky130A/libs.tech/klayout
KLAYOUT_PY_SITE : /home/<user>/klayout_gf7/lib/python3.12/site-packages
PDKPATH        : /usr/local/share/pdk/sky130A
OK klayout
OK magic
OK netgen
OK PDKPATH exists
OK Magic rcfile
OK gdsfactory
```

Doctor item	What I check
SCRIPT	Python script path loaded by KLayout batch mode.
KLAYOUT_PATH	SKY130 KLayout technology path.
KLAYOUT_PY_SITE	Must point to ~/klayout_gf7, not ~/.local.
PDKPATH	SKY130 PDK directory.
klayout	KLayout binary is available.
magic	Magic binary is available.
netgen	Netgen binary is available for optional LVS helper.
gdsfactory	Correct gdsfactory package is importable from gf7.

If `KLAYOUT\_PY\_SITE` points to `.local`, fix the launcher before changing Python code.

Command syntax

General command form:

```
skyspice2klayout_all_magicgr_analogproplus [options] <input.spice> <output.gds>
```

Common options:

Option	Values	Meaning
--doctor	none	Print environment and path checks without generating layout.
--wizard	none	Run the interactive terminal UI.
--backend	gdsfactory, magic, hybrid	Choose MOS generation strategy.
--placement	analogpro, schematic	Choose placement strategy.
--ratlines	off, points, full, both	Choose whether to generate separate ratline reference GDS files.
--flat	none	Use flat output behavior if supported by the launcher/script version.

Frequently used environment variables:

Variable	Typical value	Purpose
SKY_UNIT_ARRAY_MODE	plan or layout	Plan unit arrays or physically replace large devices with unit-array wrappers.
SKY_GUARD_RING	yes or no	Enable selected guard-ring handling.
SKY_GR_INSTS	1 2 3 4 or all	Select which MOS devices receive Magic guard-ring generation.
SKY_MAGIC_ALL_GUARD_RINGS	1	Force all supported MOS devices into Magic guard-ring mode.
SKY_CC_MODE	off, plan, markers, layout_hint	Common-centroid planning behavior.
SKY_DUMMY_MODE	marker or off	Dummy planning marker behavior.
SKY_RATLINE_GDS	off, points, full, both	Ratline reference GDS generation.
SKY_MAGIC_DRC_CHECK	1	Run Magic DRC smoke check helper.
SKY_LVS_CHECK	1	Run Magic extract/Netgen LVS helper if available.

Run folder behavior

Every normal run creates a separate output folder under the current working directory:

```
./skyspice_runs/<output_name>_<timestamp>/
```

Example:

```
./skyspice_runs/test_full_visual_20260621_204003/
```

This keeps the project folder clean. The command can be run repeatedly without overwriting older reports unless the exact output behavior is changed manually.

File group	Examples	Purpose
Main layout	<name>.gds	Clean layout view that I open in KLayout.
Ratline views	<name>_ratpoints.gds, <name>_ratlines_full.gds	Optional static connectivity reference views.
Reports	.report.json, .device_map.csv, .routing_priority.csv, .topology.json	Debugging and analysis outputs.
Run metadata	manifest.json, README_run.md, rerun.sh	Reproducibility and run tracking.
Dashboard	index.html	Quick browser view of important run files.

If I only want a clean main output without extra ratline files, I use `--ratlines off`.

Run modes

MOS backend modes

Backend	What it does	When I use it
gdsfactory	All MOS devices use the SKY130 Python/gdsfactory direct-draw path.	To test the Python draw helper flow and normal MOS generation.
magic	Selected guard-ring MOS use Magic. In v12, it should not blindly guard-ring everything unless all-guard mode is forced.	When I want Magic-generated guard-ring devices.
hybrid	Selected guard-ring MOS use Magic; remaining MOS use gdsfactory.	My usual mode. It balances Magic guard-ring generation with gdsfactory direct draw.

Placement modes

Placement mode	Intuition	What it tries to do
analogpro	More layout-aware and topology-aware.	Uses detected topology hints for mirrors, stacks, reference branches, large devices, and analog grouping.
schematic	Simpler and easier to compare with the input netlist.	Follows SPICE/netlist order more directly and keeps larger devices close together in one row.

Ratlines and ratpoints

Ratlines are debug/reference GDS geometry on non-fabrication layers. They are not real routed metal.

Option	Output
off	Only the main clean GDS.
points	Main GDS plus terminal marker GDS.
full	Main GDS plus full connected ratline GDS.
both	Main GDS plus both point markers and full ratlines.

Expected ratline files:

File	Use
<name>.gds	Main clean layout.
<name>_ratpoints.gds	Terminal point markers only. Useful when full ratlines are too visually noisy.
<name>_ratlines_full.gds	Static non-fab connectivity lines. Closest thing to a simple KiCad-style ratsnest view.

Limitations:

Limitation	Meaning
Not routed metal	The ratline GDS is visual reference geometry only.
Not live	If I move devices manually, ratlines do not update automatically.
Not DRC/LVS geometry	These are non-fab debug/reference layers.

Unit arrays, common-centroid markers, and dummy markers

Feature	Command/setting	What it means
Unit-array plan	SKY_UNIT_ARRAY_MODE=plan	Writes unit-array reports but does not replace physical MOS layout. Best for initial verification.
Unit-array layout	SKY_UNIT_ARRAY_MODE=layout	Replaces large MOS devices with unit-array wrapper cells. Useful for layout exploration.
Unit width	SKY_UNIT_W_UM=1	Target unit width for splitting large MOS devices.
Minimum W for unit array	SKY_UNIT_ARRAY_MIN_W_UM=8	Only devices with W >= this value are considered for unit-array conversion.
Common-centroid markers	SKY_CC_MODE=markers	Adds non-fab matching/common-centroid planning markers.
Dummy markers	SKY_DUMMY_MODE=marker	Adds non-fab dummy-device planning markers. Does not create real dummy MOS devices.

I usually test with `plan` first because extraction may see a physical unit array as many parallel MOS devices instead of one large MOS.

Wizard mode

Run:

```
skyspice2klayout_all_magicgr_analogproplus --wizard BGR_sky130_ckt.spice test_wizard.gds
```

The wizard asks only the main choices. Everything else stays at the script defaults.

Wizard prompt	Options	Meaning
Run profile	quick, pro, verify, clean	Controls how much planning and verification helper logic is enabled.
MOS backend	magic, gdsfactory, hybrid	Chooses how MOS devices are generated.
Placement style	analogpro, schematic	Chooses topology-aware placement or netlist-order placement.
Physical unit arrays	y, n	Controls whether large MOS devices are physically replaced by unit-array wrappers.
Common-centroid mode	off, plan, markers, layout_hint	Controls common-centroid planning files and visible marker geometry.
Dummy mode	marker, off	Adds or disables dummy planning markers.
Magic DRC	y, n	Runs Magic DRC helper when enabled.
LVS helper	y, n	Attempts Magic extraction and Netgen comparison if setup supports it.
Ratline GDS	off, points, full, both	Generates optional ratline reference files.
Guard-ring selection	instance numbers, all, n	Selects which MOS devices get Magic guard-ring generation.

Guard-ring prompt example:

```
1) X1 ...  
2) X2 ...  
3) X3 ...  
4) X4 ...
```

Add guard rings to selected FETs? [y/N or instance numbers]:

For the BGR test case, I can type:

```
1 2 3 4
```

or type `y`, then enter `1 2 3 4` at the next prompt.

Command examples

1. Basic working folder

```
cd ~/Analogprov12plus_runs  
ls
```

Expected:

```
required_docs  
required_scripts  
skyspice_runs  
BGR_sky130_ckt.spice
```

2. gdsfactory backend test

This checks whether the gf7/gdsfactory path is working.

```
SKY_UNIT_ARRAY_MODE=plan \  
skyspice2klayout_all_magicgr_analogproplus \  
--backend gdsfactory \  
--placement analogpro \  
--ratlines off \  
BGR_sky130_ckt.spice \  
test_gdsfactory_fixed.gds
```

Expected idea:

```
Direct draw used: 18  
Fixed GDS used : 1  
Missing      : 0
```

If this fails, it is usually a gf7/path/API issue, not a SPICE parsing issue.

3. Hybrid selected guard-ring test

This is my main useful mode.

```
SKY_UNIT_ARRAY_MODE=plan \  
SKY_GUARD_RING=yes \  
SKY_GR_INSTS="1 2 3 4" \  
skyspice2klayout_all_magicgr_analogproplus \  
--backend hybrid \  
--placement analogpro \  
--ratlines off \  
BGR_sky130_ckt.spice \  
test_hybrid_fixed.gds
```

Expected behavior:

```
X1 X2 X3 X4 -> Magic guarded MOS  
X5 onward  -> gdsfactory direct draw  
XQ0        -> fixed GDS  
Missing    -> 0
```

4. Magic selected-only test

In v12, `--backend magic` is selection-safe. It should not blindly guard-ring every MOS unless I explicitly ask for that.

```
SKY_UNIT_ARRAY_MODE=plan \  
SKY_GUARD_RING=yes \  
SKY_GR_INSTS="1 2 3 4" \  
skyspice2klayout_all_magicgr_analogproplus \  
--backend magic \  
--placement analogpro \  
--ratlines off \  
BGR_sky130_ckt.spice \  
test_magic_selected_only.gds
```

Expected behavior:

```
Only X1 X2 X3 X4 get Magic guard-ring generation.  
Other MOS devices use the gf7/gdsfactory path.
```

If I intentionally want every MOS generated through Magic guard-ring mode:

```
SKY_MAGIC_ALL_GUARD_RINGS=1 \  
SKY_GUARD_RING=yes \  

```

```
SKY_GR_INSTS=all \
skyspice2klayout_all_magicgr_analogproplus \
--backend magic \
--placement analogpro \
--ratlines off \
BGR_sky130_ckt.spice \
test_magic_all_guarded.gds
```

I do not use this as the default because it can make the layout bulky and guard-ring heavy.

5. Schematic placement test

```
SKY_UNIT_ARRAY_MODE=plan \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement schematic \
--ratlines both \
BGR_sky130_ckt.spice \
test_schematic_placement.gds
```

6. Ratline and ratpoint test

```
SKY_UNIT_ARRAY_MODE=plan \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines both \
BGR_sky130_ckt.spice \
test_ratlines.gds
```

Open the generated files:

```
klayoutpdkfull skyspice_runs/*test_ratlines*/test_ratlines.gds
klayoutpdkfull skyspice_runs/*test_ratlines*/test_ratlines_ratpoints.gds
klayoutpdkfull skyspice_runs/*test_ratlines*/test_ratlines_ratlines_full.gds
```

7. Full visual test command

This turns on the main visual/planning features.

```
SKY_UNIT_ARRAY_MODE=layout \
SKY_UNIT_W_UM=1 \
SKY_UNIT_ARRAY_MIN_W_UM=8 \
SKY_GUARD_RING=yes \
SKY_GR_INSTS="1 2 3 4" \
SKY_CC_MODE=markers \
SKY_DUMMY_MODE=marker \
SKY_RATLINE_GDS=both \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines both \
BGR_sky130_ckt.spice \
test_full_visual.gds
```

Expected outputs:

```
skyspice_runs/test_full_visual_<timestamp>/
├─ test_full_visual.gds
├─ test_full_visual_ratpoints.gds
├─ test_full_visual_ratlines_full.gds
├─ test_full_visual.gds.report.json
├─ test_full_visual.gds.device_map.csv
├─ test_full_visual.gds.routing_priority.csv
├─ test_full_visual.gds.topology.json
└─ test_full_visual.gds.unit_arrays.csv
```

```
├─ index.html
├─ manifest.json
├─ README_run.md
└─ rerun.sh
```

8. Optional Magic DRC smoke check

```
SKY_UNIT_ARRAY_MODE=plan \
SKY_GUARD_RING=yes \
SKY_GR_INSTS="1 2 3 4" \
SKY_MAGIC_DRC_CHECK=1 \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines off \
BGR_sky130_ckt.spice \
test_drc_check.gds
```

This writes a log like:

```
test_drc_check.gds.magic_drc.log
```

I treat this as a smoke check, not final signoff.

9. Full verification helper command

This command keeps unit arrays in `plan` mode because physical unit-array replacement can confuse strict LVS unless device-summing is handled.

```
SKY_MOS_BACKEND=hybrid \
SKY_GUARD_RING=yes \
SKY_GR_INSTS="1 2 3 4" \
SKY_UNIT_ARRAY_MODE=plan \
SKY_CC_MODE=markers \
SKY_DUMMY_MODE=marker \
SKY_RATLINE_GDS=both \
SKY_MAGIC_DRC_CHECK=1 \
SKY_LVS_CHECK=1 \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines both \
BGR_sky130_ckt.spice \
verify_full.gds
```

Expected useful files if the local setup supports the helper flow:

```
verify_full.gds
verify_full.gds.report.json
verify_full.gds.magic_drc.log
verify_full.gds.magic_extract_lvs.log
verify_full.gds.netgen_lvs.log
```

I treat this as a helper check. It does not replace proper manual DRC/LVS review.

10. Open generated outputs

Open the main GDS:

```
klayoutpdkfull skyspice_runs/*test_full_visual*/test_full_visual.gds
```

Open the ratpoint view:

```
klayoutpdkfull skyspice_runs/*test_ratlines*/test_ratlines_ratpoints.gds
```

Open the full ratline view:

```
klayoutpdkfull skyspice_runs/*test_ratlines*/test_ratlines_ratlines_full.gds
```

Open the dashboard:

```
xdg-open skyspice_runs/*test_full_visual*/index.html
```

11. Wizard run

```
skyspice2klayout_all_magicgr_analogproplus \  
--wizard \  
BGR_sky130_ckt.spice \  
test_wizard.gds
```

Generated outputs

Output	What I use it for
<name>.gds	Main clean generated layout.
<name>_ratpoints.gds	Terminal point reference GDS.
<name>_ratlines_full.gds	Static non-fab ratline reference GDS.
<name>.gds.report.json	Main run summary.
<name>.gds.device_map.csv	Device instance, model, method, nets, parameters, and raw SPICE info.
<name>.gds.nets.json	Net summary.
<name>.gds.routing_priority.csv	Important nets and routing hints.
<name>.gds.topology.json	Detected analog topology groups.
<name>.gds.layout_warnings.csv	Layout cautions and review notes.
<name>.gds.unit_arrays.csv	Unit-array planning/replacement summary.
index.html	Run dashboard.
manifest.json	Machine-readable run metadata.
README_run.md	Run-specific notes.
rerun.sh	Reproducible command for that run.

Useful inspection commands:

```
xdg-open skyspice_runs/*test_full_visual*/index.html  
  
jq '.placed_total, .missing_total, .placement_summary, .ratlines, .unit_arrays' \  
skyspice_runs/*test_full_visual*/test_full_visual.gds.report.json  
  
column -s, -t < skyspice_runs/*test_full_visual*/test_full_visual.gds.device_map.csv | less -S  
  
column -s, -t < skyspice_runs/*test_full_visual*/test_full_visual.gds.routing_priority.csv | less -S
```

Troubleshooting

Problem	Likely reason	What I check first
KLAYOUT_PY_SITE shows .local	Launcher is not forcing gf7.	Patch the launcher. Do not change placement logic first.
dev_temp already exists	Helper cells created repeated internal names.	Use the gf7-compatible launcher/script and confirm the active launcher path.
add_array, spacing, or move errors	gdsfactory/KFactory API mismatch.	Confirm ~/klayout_gf7 and gdsfactory 7.x.
.spice opens as a layout stream	Launcher passed SPICE directly to KLayout.	Launcher should convert args to SKY_NETLIST and SKY_OUT.
Unknown option: --	KLayout version rejects extra --.	Use a launcher that avoids passing -- to KLayout.

Magic works but hybrid/gdsfactory fails	Magic path is healthy, gdsfactory path is not.	Run --doctor, then test gdsfactory-only.
Ratlines stay static after moving devices	Ratlines are not live.	Regenerate ratlines after changing placement.

Doctor check again:

```
skyspice2klayout_all_magicgr_analogproplus --doctor
```

Good path:

```
KLAYOUT_PY_SITE : /home/<user>/klayout_gf7/lib/python3.12/site-packages
```

Bad path:

```
KLAYOUT_PY_SITE : /home/<user>/local/lib/python3.12/site-packages
```

Validation order

Run these in order after a fresh setup or after moving the project to another machine:

```
# 1. Check paths
skyspice2klayout_all_magicgr_analogproplus --doctor

# 2. Test gdsfactory direct draw
SKY_UNIT_ARRAY_MODE=plan \
skyspice2klayout_all_magicgr_analogproplus \
--backend gdsfactory \
--placement analogpro \
--ratlines off \
BGR_sky130_ckt.spice \
test_gdsfactory_fixed.gds

# 3. Test hybrid selected guard rings
SKY_UNIT_ARRAY_MODE=plan \
SKY_GUARD_RING=yes \
SKY_GR_INSTS="1 2 3 4" \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines off \
BGR_sky130_ckt.spice \
test_hybrid_fixed.gds

# 4. Test schematic placement
SKY_UNIT_ARRAY_MODE=plan \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement schematic \
--ratlines both \
BGR_sky130_ckt.spice \
test_schematic_placement.gds

# 5. Test ratline files
SKY_UNIT_ARRAY_MODE=plan \
skyspice2klayout_all_magicgr_analogproplus \
--backend hybrid \
--placement analogpro \
--ratlines both \
BGR_sky130_ckt.spice \
test_ratlines.gds

# 6. Test wizard UI
skyspice2klayout_all_magicgr_analogproplus \
--wizard \
BGR_sky130_ckt.spice \
test_wizard.gds
```

Once these pass, I consider the local setup usable.

Final note

This is not meant to replace real analog layout skill. I made it because I wanted a cleaner, more useful SPICE-to-KLayout starting point than the rough native import experience I was fighting with.

It is a playground project, but it became useful enough that I wanted the setup to be reproducible. If something breaks, I first check the launcher paths, then the gf7 environment, then the PDK helper paths. I do not randomly change the working Python flow unless the path and environment checks are clean.